

정규 표현식 레포트

2603340032 남종희

1. 정규 표현식 문법 및 패턴

1.1 문자 클래스

[abc] : a, b, c중에 하나와 매치 ex) before 에서 'b' 매치

[a-z] / [A-Z] : 알파벳 소문자 / 알파벳 대문자 범위

[0-9] : 숫자 전체 범위

[가-힣] : 한글 전체 범위

[a-zA-Z] : 알파벳 대소문자 범위

[a-zA-Z0-9] : 알파벳 대소문자 및 숫자 범위

[^0-9] : 숫자 제외한 모든 문자(제외의 의미)

[0-9^] : 숫자전체와 특수문자 ^ 범위

[a-zA-Z0-9_]: 알파벳 대소문자 및 숫자, 공백 범위

예시) "Hello123 코딩"

[^0-9] : "H", "e", "l", "l", "o", " ", "코", "딩"

[가-힣] : "코", "딩"

1.2 단축 기호

\d : 숫자 [0-9] 와 같다

\D : 숫자 제외한 모든 문자 [^0-9] 와 같다

\s : 공백, 탭, 줄바꿈 등 화이트스페이스

\S : 공백이 아닌 것

\w : 단어 문자(알파벳 + 숫자 + _) [a-zA-Z0-9_] 와 같다

\W : 단어 문자가 아닌 것 [^a-zA-Z0-9_] - 주로 기호나 간격을 찾을 때(_ 제외)

1.3 메타 문자 및 수량자

. : 줄바꿈을 제외한 모든 문자 1개와 매치

* (0회 이상 반복) / + (1회 이상 반복) / ? (0회 또는 1회)

{n, m} : 최소 n번에서 최대 m번까지 반복

^ / \$: 문자열의 시작 / 문자열의 끝

예시)

a.b : a와 b사이에 어떤 글자가 와도 되니 aab, abb, a#b, a b(공백도 한 글자)

apples? : s가 0개 혹은 1개 이므로 apple, apples 모두 찾아냄

ca*t : a가 0회이상 반복하면 되니 ct, caat 모두 찾아냄

d+og : d가 1회이상 반복하면 되니 dog, ddog 모두 찾아냄 (og는 X)

\d{3}-\d{4} : "010-1234" -포함 정확한 자릿수 숫자 검증

1.4 그룹핑

(abc)+ : "abc"가 1번 이상 반복되는것을 매칭

- 그룹 지정 후 데이터 추출하기

```
user_info = "park 010-1234-5678"
```

```
m = re.search(r"(\w+)\s+(\d+-\d+-\d+)", user_info)
```

```
print(f"전체 문장: {m.group(0)}")
```

```
print(f"추출된 이름: {m.group(1)}")
```

```
print(f"추출된 번호: {m.group(2)}")
```

2. re 모듈 메소드

re.match(pattern, string)

- 문자열의 처음부터 패턴이 일치하는지 확인
- 문자열 전체가 특정 형식을 따르는지 검증할 때 유용
- 예시) 아이디 형식 확인

re.search(pattern, string)

- 문자열 전체를 검색하여 패턴과 일치하는 부분이 있는지 확인
- 긴 텍스트 안에서 특정 키워드나 패턴을 찾아낼 때 사용

`re.findall(pattern, string)`

- 패턴과 일치하는 모든 문자열을 찾아 리스트로 변화
- 본문 내 모든 이메일 주소 추출, 모든 숫자 추출

`re.split(pattern, string)`

- 패턴을 구분자로 하여 문자열을 나눔
- 단순 쉼표뿐만 아니라 공백, 마침표 등 복잡한 기준으로 문자열을 쪼갤 때 유용

`re.sub(pattern, replace, string)`

- 패턴과 일치하는 부분을 다른 문자열로 교체
- 데이터 정제

예시) `text = "Contact: park 010-1234-5678, kim 010-9876-5432"`

`pattern`에서 `r(Raw string)`을 쓰는 이유는 백슬래시를 특수기호로 인식하지않고 있는 그대로 읽게 하기 위해서 사용한다.

`re.compile` 을 사용하면 패턴을 저장해두고 반복적으로 사용할때 유용하게 사용가능하다.

1.1) `re.match(pattern, string)` “Contact로 시작 하는지 확인”

```
m = re.match(r"Contact", text)
```

```
print(m.group()) # Contact
```

1.2) `p = re.compile(r"Contact")` “컴파일 사용”

```
m = p.match(text)
```

```
print(m.group()) # Contact
```

2.1) `re.search(pattern, string)` “문장 중 첫 번째 전화번호 추출”

```
m = re.search(r"\d+-\d+-\d+", text)
```

```
print(m.group()) # 010-1234-5678
```

2.2) `p = re.compile(r"\d+-\d+-\d+")` “컴파일 사용”

```
m = p.search(text)
```

```
print(m.group()) # 010-1234-5678
```

3.1) re.findall(pattern, string) “문장 내 모든 전화번호 리스트로 추출”

```
numbers = re.findall(r"\d+-\d+-\d+", text)
print(numbers) # ['010-1234-5678', '010-9876-5432']
```

3.2) p = re.compile(r"\d+-\d+-\d+") “컴파일 사용”

```
numbers = p.findall(text)
print(numbers) # 결과: ['010-1234-5678', '010-9876-5432']
```

4.1) re.split(pattern, string) “쉼표, 콜론, 공백이 하나 이상 반복되는 부분 기준으로 분리”

```
result = re.split(r"[:\s]+", text)
print(result) # ['Contact', 'park', '010-1234-5678', 'kim', '010-9876-5432']
```

4.2) p = re.compile(r"[:\s]+") “컴파일 사용”

```
result = p.split(text)
print(result) # ['Contact', 'park', '010-1234-5678', 'kim', '010-9876-5432']
```

5.1) re.sub(pattern, replace, string) “전화번호 부분을 * 처리하기”

```
result = re.sub(r'\d', '*', text)
print(result) # Contact: park ***-****-****, kim ***-****-****
```

5.2) p = re.compile(r'\d') “컴파일 사용”

```
result = p.sub('*', text)
print(result) # Contact: park ***-****-****, kim ***-****-****
```